

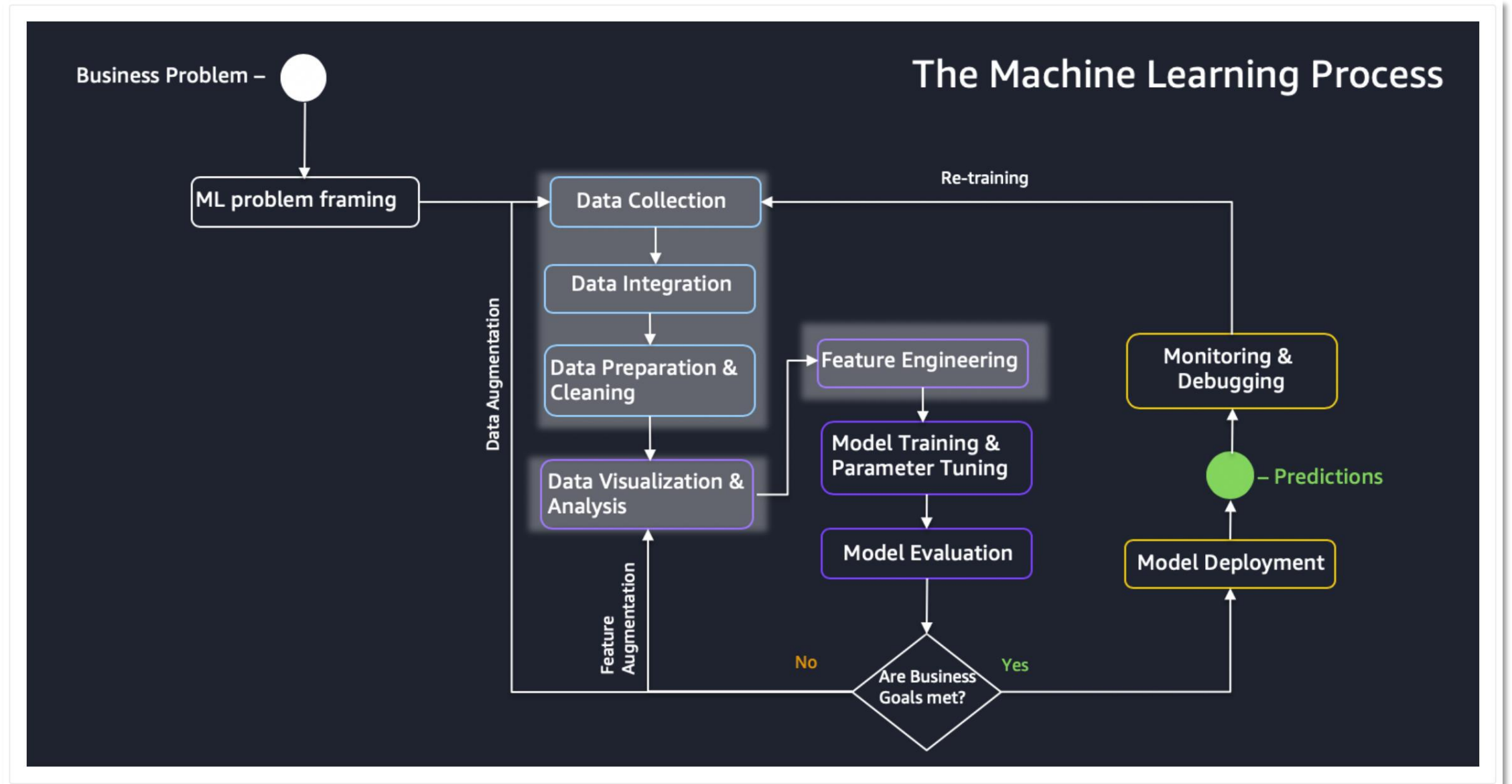
Module 2: Intelligent Data Driven Applications on the Cloud

INTELLIGENT DATA DEVELOPMENT, JUNE 2026

HANDS ON SESSION

INTELLIGENT DATA DEVELOPMENT, JUNE 2026

CLOUD MACHINE LEARNING



HANDS-ON CLOUD LAB: STEPS

- ~~1. Connect to an AWS account using your IAM user – 10 mins~~
- ~~2. Create an S3 bucket like idd-class-folder and load data.csv – 10 mins~~
- ~~3. Delete the S3 bucket and empty it – 5 mins~~
4. Create a Lambda function by hand – 20 mins
5. Link the Lambda to S3 and give it proper permissions – 15 mins
6. Load data from s3 and save data in csv format into s3 – 20 mins
7. Launch all above with a CloudFormation template – 15 mins

HANDS ON – STEP 1 - CONNECT TO AN AWS ACCOUNT USING AN IAM USER

Step-by-step:

1. Go to the login link: **<https://idd-class.signin.aws.amazon.com/console>**
2. Log in using the credentials:
 - **Username:** *name.surname/*
name.middlename (or your assigned name)
 - **Password:** *ChangeMeNow_2024!*
3. On first login, set a **new password** you'll remember. A password reset screen will appear. You will need to add the old password provided and add your new one.

You're now inside your AWS account!

The image displays two screenshots from the AWS IAM console. The left screenshot, titled 'IAM user sign in', shows the login interface with fields for 'Account ID or alias' (containing 'idd-class'), 'IAM username' (containing 'idd2026student1'), and 'Password'. It includes a 'Remember this account' checkbox, a 'Show Password' checkbox, and buttons for 'Sign in' and 'Sign in using root user email'. A link for 'Having trouble?' is also present. The right screenshot, titled 'Password reset', shows the password reset interface. It includes a message about password resets being managed by the administrator, a list of password requirements (minimum 8 characters, mix of uppercase, lowercase, numbers, and special characters, and not identical to the AWS account name or email address), and fields for 'Old Password', 'New Password', and 'Confirm New Password'. It also features 'Show Password' checkboxes, a 'Matches' indicator, and buttons for 'Confirm Password Change' and 'Sign in to a different account'.

IAM user sign in ⓘ

Account ID or alias [\(Don't have?\)](#)

idd-class

☐ Remember this account

IAM username

idd2026student1

Password

.....

☐ Show Password [Having trouble?](#)

Sign in

Sign in using root user email

[Create a new AWS account](#)

Password reset ⓘ

Password resets are managed by your administrator. In most cases your password should contain:

- Minimum password length is 8 characters
- Include a minimum of three of the following mix of character types: uppercase, lowercase, numbers, and ! @ # \$ % ^ & * () _ + - = [] { } | ' + - = [] { } | '
- Must not be identical to your AWS account name or email address

Your account (223947838) expired or requires a reset

To continue, please verify new password for student

Old Password

.....

☐ Show Password

New Password

.....

Confirm New Password

.....

☐ Show Password Matches

Confirm Password Change

[Sign in to a different account](#)

HANDS ON – STEP 2 – UPLOAD OUR DATA TO OUR S3 BUCKET

We'll add the train_data.csv to our own S3 buckets.

Steps:

1. Search for **S3** in the top search bar, click **S3**
2. Search for **<name>-<surname>-bucket-idd**, i.e. **laura-cozma-bucket-idd**
3. Upload data_train.csv

HANDS ON – STEP 3 - CREATE A LAMBDA FUNCTION BY HAND

Step 1: Create an IAM Role for Lambda

1. Go to **IAM > Roles**
2. Click **Create role**
3. Choose **AWS service** → **Lambda**
4. Attach the following policies:
 1. AmazonS3FullAccess (or scoped-down access)
 2. AWSLambdaBasicExecutionRole
5. Name the role (e.g. LambdaS3Role) and create it.

Step 2: Create the Lambda Function

1. Go to **Lambda** → **Create function**
2. Choose:
 1. Name: ForecastingFunction
 2. Runtime: **Python 3.X**
 3. Execution role: Use existing role → select LambdaS3PandasRole
3. Create the function
4. Go to Configuration and set up time out more than 3 seconds.
5. Go to Layers and choose Pandas from the Layer's list.

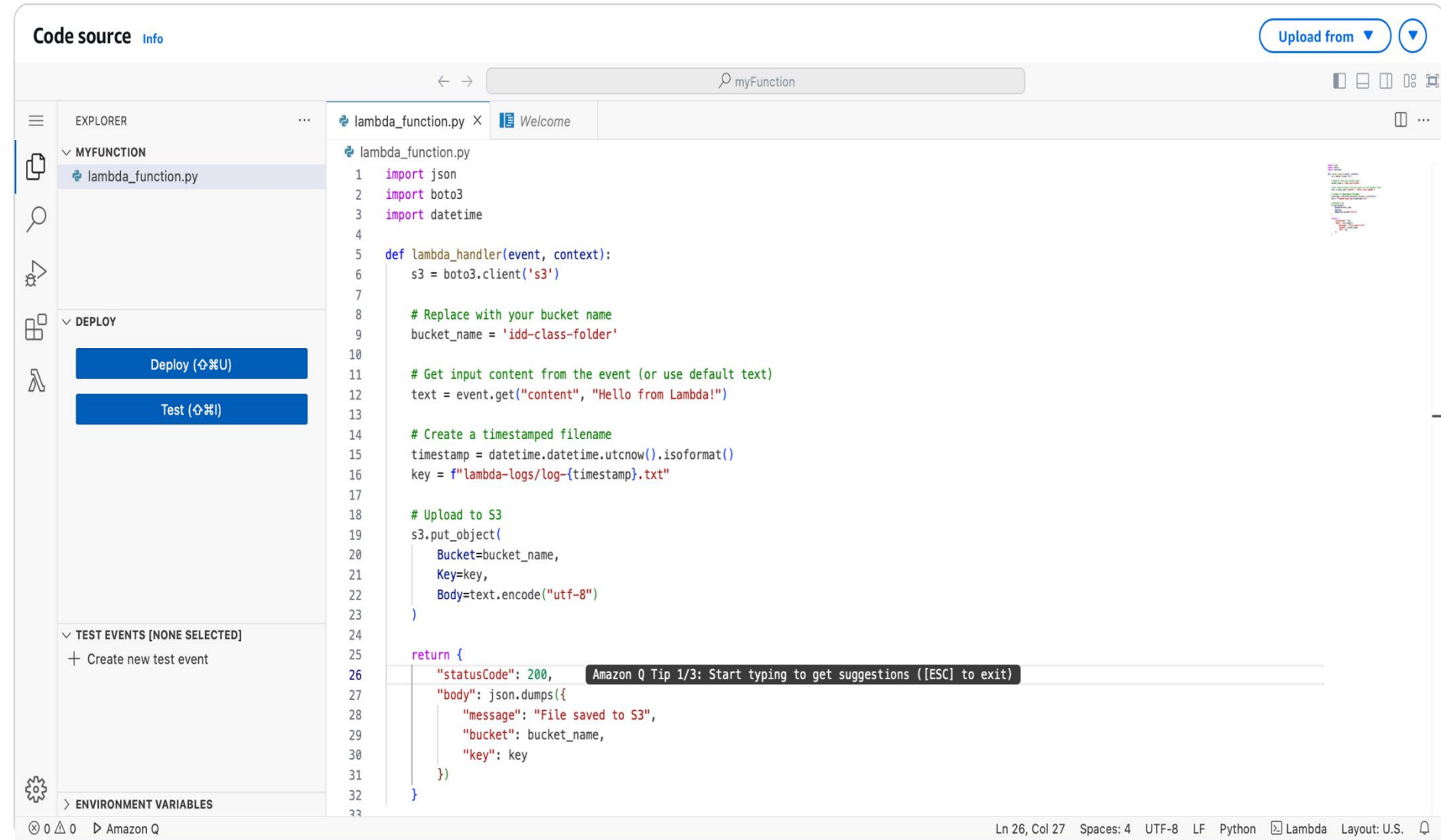
HANDS ON – STEP 3 - CREATE A LAMBDA FUNCTION BY HAND

Step 3: Edit Lambda Code

Click **Code** and paste the following:
Replace your-input-bucket-name and file keys with your actual S3 path.

Step 4: Test the Lambda

1. Click **Test** → Create a test event (can be empty if hardcoded paths are used)
2. Run the Lambda
3. Check your S3 bucket to confirm the new file is saved



The screenshot displays the AWS Lambda console's code editor interface. The left sidebar shows the 'EXPLORER' view with a file named 'lambda_function.py' under the 'MYFUNCTION' folder. Below this, the 'DEPLOY' section contains 'Deploy' and 'Test' buttons. The main editor area shows the Python code for the lambda function. The code imports 'json', 'boto3', and 'datetime', defines a 'lambda_handler' function, and includes comments for replacing bucket names and creating timestamps. It uses 's3.put_object' to upload the event content to an S3 bucket. The status bar at the bottom indicates the file is at line 26, column 27, using UTF-8 encoding and LF line endings.

```
Code source Info
Upload from
myFunction
lambda_function.py X Welcome
lambda_function.py
1 import json
2 import boto3
3 import datetime
4
5 def lambda_handler(event, context):
6     s3 = boto3.client('s3')
7
8     # Replace with your bucket name
9     bucket_name = 'add-class-folder'
10
11     # Get input content from the event (or use default text)
12     text = event.get("content", "Hello from Lambda!")
13
14     # Create a timestamped filename
15     timestamp = datetime.datetime.utcnow().isoformat()
16     key = f"lambda-logs/log-{timestamp}.txt"
17
18     # Upload to S3
19     s3.put_object(
20         Bucket=bucket_name,
21         Key=key,
22         Body=text.encode("utf-8")
23     )
24
25     return {
26         "statusCode": 200,
27         "body": json.dumps({
28             "message": "File saved to S3",
29             "bucket": bucket_name,
30             "key": key
31         })
32     }
33
Ln 26, Col 27 Spaces: 4 UTF-8 LF Python Lambda Layout: U.S.
```


HANDS ON – STEP 3 - CREATE A LAMBDA FUNCTION BY HAND

Step 5: Package pandas for Lambda - optional

Lambda has a size limit for inline editing, and pandas isn't included by default. You must **deploy with a layer** or **zip upload**.

Option A: Use a Prebuilt Lambda Layer (Recommended for Console Users)

Use a public pandas layer:

1. Visit [AWS Lambda Layer Library](#) → Find the **pandas layer ARN** for your region (e.g. arn:aws:lambda:us-east-1:770693421928:layer:Klayers-p39-pandas:1)
2. Save the ARN for later

Step 6: Add pandas Layer to Lambda - optional

1. Open the Lambda function → Click **Layers** → **Add a layer**
2. Choose **Specify an ARN**
3. Paste your pandas layer ARN and click **Add**

LAMBA CODE

```
import json
import boto3
import datetime
import pandas as pd
from io import StringIO

def lambda_handler(event, context):
    s3 = boto3.client('s3')

    # Replace with your actual bucket name
    bucket_name = 'name-surname-bucket-idd'
    input_key = 'train_data.csv'

    try:
        # Read CSV from S3
        response = s3.get_object(Bucket=bucket_name, Key=input_key)
        content = response['Body'].read().decode('utf-8')
        df = pd.read_csv(StringIO(content))

        # Do any transformation here (this is just a placeholder)
        df_processed = df.copy()

        # Prepare to write back to S3
        timestamp = datetime.datetime.utcnow().isoformat()
        output_key = f"processed/output_{timestamp}.csv"
        csv_buffer = StringIO()
        df_processed.to_csv(csv_buffer, index=False)

        # Upload to S3
        s3.put_object(
            Bucket=bucket_name,
            Key=output_key,
            Body=csv_buffer.getvalue().encode('utf-8')
        )
```

```
        return {
            "statusCode": 200,
            "body": json.dumps({
                "message": "CSV processed and saved to S3",
                "bucket": bucket_name,
                "output_key": output_key
            })
        }

    except Exception as e:
        return {
            "statusCode": 500,
            "body": json.dumps({
                "error": str(e)
            })
        }
```

Things Change. Know Why - Key Use Case Examples

INTELLIGENT DATA DEVELOPMENT, JUNE 2026

INVITED SPEAKER(S)



OBRYAN POYSER

DATA SCIENCE LEAD
@PEPSICO



CARLOS RANGEL

ML LEAD
@PEPSICO



MIRIAM POTAU

DATA SCIENTIST
@PEPSICO



PRIYA PATHAK

ML ENGINEER
@PEPSICO

THANK YOU!